



University of Sussex
Engineering & Informatics

Candidate Number: 282953

Name: Ho Chit Joure

BSc Computer Science and Artificial Intelligence
Department of Informatics

Timeline-Based Planning

Supervisor: Dr Hsi-Ming Ho

2026

Statement of Originality

This report is submitted as part requirement for the degree of **BSc Computer Science and Artificial Intelligence** at the University of Sussex. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged. I hereby **give / withhold** permission for a copy of this report to be loaned out to students in future years **(delete as necessary)**.

Acknowledgements

I would to thank my supervisor Dr. His-Ming Ho for his help, support and useful ideas throughout the graduation project. His comments help me to provide more technical focus and academic grounding to my dissertation.

I am also appreciative to the Department of Informatics, University of Sussex for giving me the information, resources and the learning environment that I needed to accomplish my research successfully. “School gave me the knowledge and skills that were a critical base for my profession.”

I also thank the availability of open-source tools and research materials, especially the Z3 SMT solver, which was key to the success of the project. I would also want to acknowledge the wider research community whose work in the fields of autonomous systems, automated planning and artificial intelligence has been an inspiration for my thesis.

Finally I would to thank my family and friends for their support, patience and encouragement during my academic path. Main reason to carry out this task was their understanding and support.

Professional Considerations

The project was the design and development of a time-based planning system for continuous-time scheduling utilising Satisfiability Modulo Theories (SMT). This experiment did not include human participants, personal data, medical data or any direct user testing needing ethics clearance. As a result, no formal ethical review application was required for the project. Nevertheless, professional and ethical considerations remained important throughout the design, implementation, and reporting process.

A key professional consideration concerned academic integrity and the responsible use of sources. Throughout the project, published research papers, technical documentation and internet information were utilised to support the theoretical basis, modelling judgements and implementation choices. All of these items are properly cited and attributed. This is an academic honesty issue, but it is also a professional ethics issue in computers, where openness and credit are valued.

Another crucial aspect was the dependability of the system. The project focuses on autonomous planning for a Mars rover, where errors in scheduling might result in lost resources or inability to operate. This dissertation describes a simplified prototype, not a deployable mission system. Yet the work was done with the realisation the planning systems in real-world domains must be reliable and rigorously evaluated. For this reason, the study focused on formal constraint modelling and SMT-based reasoning, which provide greater accuracy guarantees than merely heuristic techniques.

The project also required consideration of model limitations and responsible interpretation of results. The planning framework developed here is an abstraction of real rover operations. It does not fully capture all mission factors, such as uncertain terrain, communication delay, hardware degradation, or detailed environmental change. Professionally, it is therefore important not to overstate the capabilities of the prototype. The results presented in this report should be understood as evidence of the feasibility of the proposed approach within a simplified research setting, rather than as proof of direct readiness for real operational deployment.

A further professional consideration concerns software engineering practice. The implementation is modular for better clarity, maintainability and repeatability. Input management, constraint formation, interaction with the solver and display of the results were divided into several phased. Experimental runs were also documented through recorded inputs, solver outcomes, and runtime measurements. These choices reflect good professional practice in

computing, were systems should be developed in a way that allows later verification, extension, and review.

Finally, this project has been guided by the broader principles of professional responsibility in computing, including honesty, competence, respect for evidence, and awareness of the limits of technical systems. Even though the work is primarily theoretical and experimental, these considerations remain relevant because autonomous planning systems may ultimately be applied in safety-critical and resource-constrained environments. The project therefore aims not only to contribute a technical prototype, but also to demonstrate a responsible and professionally grounded approach to research and system development.

Abstract

In this dissertation we study the application of timeline-based planning to continuous-time domains using Satisfiability Modulo Theories (SMT). This work deals with the problem of planning activities with length, temporal dependence and resource limits, which are difficult to express precisely using classical planning approaches that use discrete time. A formal planning model is developed in which tasks are represented as temporal intervals with start times, end times, and duration bounds, while constraints are encoded as linear relations over real-valued variables. The model is implemented as a prototype planning system in Python using the Z3 SMT solver.

A Mars rover scheduling scenario is used as the main case study to evaluate the proposed approach. The rover is required to perform tasks such as movement, imaging, analysis, and recharge under temporal ordering, energy, and daylight constraints. Experimental evaluation is conducted on planning instances of varying size and constraint density in order to assess feasibility, runtime, and scalability. The results show that the system can successfully generate valid schedules for small and medium-sized problems and can correctly identify infeasible cases when constraints conflict. The results also show that the solver performance drops with the rise of the number of tasks and the density of constraints, and the scalability is a big challenge.

Overall, the project demonstrates that SMT-based timeline planning provides a rigorous and expressive framework for modelling continuous-time scheduling problems in autonomous systems. Although the current prototype is simplified and not yet suitable for large-scale deployment, it offers a practical foundation for future work in optimization, uncertainty handling, and more realistic autonomous mission planning.

Table of Contents

<i>Statement of Originality</i>	2
<i>Acknowledgements</i>	3
<i>Professional Considerations</i>	4
<i>Abstract</i>	6
<i>1. Introduction</i>	8
<i>2. Literature Review</i>	10
<i>3. Problem Formulation</i>	12
<i>4. System Design and Architecture</i>	14
<i>5. Implementation</i>	18
<i>6. Algorithm Design and Analysis</i>	21
<i>7. Optimization-Based Planning Extension</i>	23
<i>8. Mars Rover Planning Scenario</i>	25
<i>9. Experimental Methodology</i>	28
<i>10. Results</i>	31
<i>11. Discussion</i>	35
<i>12. Conclusion and Future Work</i>	37
<i>References</i>	39
Appendix A: Project Log Summary	41
Appendix B: Example Input Problem Instance	42
Appendix C: Additional Experimental Results	42
Appendix D: Example Solver Output	43

1. Introduction

Planning is a fundamental problem in artificial intelligence because it concerns how an intelligent system selects and organises actions to achieve goals under give constraints. In many traditional planning approaches, time is represented as a sequence of discrete steps. This makes thinking simpler, but doesn't adequately model many real-world systems. In actuality, activities usually have a length, may overlap, and must be coordinated throughout continuous time. This is especially true in fields such as robots and autonomous systems where the timing of activities directly impacts safety, efficiency and performance of tasks.

Timeline-based planning provides a more expressive way of modelling such problems. Instead of treating actions as isolated instantaneous events, it represents system behaviour as a set of timelines made up of temporal tokens, where each token corresponds to an activity with a start time, end time, and duration. This allows the model to capture concurrency, synchronization, and complex temporal dependencies more naturally (**Cesta, Fratini and Pecora, 2009**). This makes timeline-based planning especially suitable for sectors where temporal precision is important such as robotics, industrial automation and space exploration.

Mars rover mission are a good example of the necessity for such planning systems. Autonomous rovers have to deal with navigation, imaging, data collecting, processing and energy management under severe temporal, environmental and resource restrictions. Some chores can only be done in daylight. Some tasks need batteries. Some tasks require you to have done prior jobs. Because communication delays between Earth and Mars are substantial, rovers must also operate with a high degree of autonomy. Effective planning is therefore essential for mission success.

However, continuous-time planning is computationally challenging. When time is represented using real-valued variables rather than discrete steps, the problem becomes harder to reason about, and complexity increases rapidly as the number of tasks and continuous-time reasoning together with resource and environmental conditions in a single framework. This creates a need for practical methods that are both expressive and computationally manageable.

This dissertation addresses that need through the use of Satisfiability Modulo Theories (SMT). SMT extends Boolean satisfiability to reasoning about more expressive theories, such as linear arithmetic. In this project SMT is used to model task start timings, finish times, durations and dependencies as formal constraints on real valued variables. Using the Z3 SMT solver, the dissertation

develops a prototype system for continuous-time planning and applies it to a Mars rover case study (**De Moura and Bjørner, 2008**).

The main contribution of the project is to show how timeline-based planning can be formulated, implemented, and evaluated in a continuous-time autonomous systems context. By combining formal modelling, SMT-based solving, and empirical analysis, the dissertation argues that SMT-based timeline planning is a promising approach for realistic scheduling problems, even though scalability remains an important challenge.

2. Literature Review

Timeline-based planning has become an important approach for representing and reasoning about temporal behaviour in artificial intelligence. Unlike classical planning methods, which often treat actions as instantaneous events arranged in a discrete sequence, timeline-based planning models system behaviour as a set of timelines composed of tokens. Each token represents an activity extending over a period of time and is associated with a start time, end time, and duration. It enables the planner to describe concurrency, synchronisation and temporal dependence in a more natural manner, which makes the framework especially useful for fields such as robotics, aerospace and industrial automation (**Cesta, Fratini and Pecora, 2009**).

A major issue in planning research concerns how time is represented. In discrete-time planning, time is divided into fixed steps, which simplifies computation but can reduce realism. Many real-world tasks do not begin and end on neat integer boundaries. Instead, they may occur at arbitrary points in time, overlap with one another, and have variable durations. Continuous-time planning addresses this by representing time using real-valued variables. Such models are particularly relevant in robotics and autonomous systems, where physical execution needs precise time (**Cashmore, Fox and Long, 2016**). However, this increase in realism also leads to greater computational complexity, since reasoning over continuous temporal domains is harder than reasoning over discrete state transitions.

To manage such complexity, many planning problems are expressed using constraint-based reasoning. This strategy is to state the issue in terms of variables and constraints and to discover values that meet all the requirements. In the context of timeline-based planning these variables generally reflect job start timings, finish times and durations, while the constraints indicate temporal ordering, synchronisation, resource limits and execution windows. Constraint-based modelling is interesting in that it is declarative: rather than defining how a schedule is to be found, it states the requirements that any valid schedule must fulfil. It gives a versatile and accurate approach for complex temporal planning.

In this framework, Satisfiability Modulo Theories (SMT) has gained great relevance. SMT generalises Boolean satisfiability to enable formulae over broader mathematical theories, e.g. linear arithmetic. This allows SMT solvers to deal with logical structure and numerical constraint together in a single framework (**Barrett et al., 2009**). Such capacity is especially important for continuous-time planning, where temporal interactions may be expressed as linear inequalities on real-valued variables. Among available SMT solvers, Z3 is one of the most widely used. Developed by Microsoft Research, Z3 supports

linear real arithmetic and is therefore well suited to modelling task durations, temporal ordering, and resource constraints (**De Moura and Bjørner, 2008**). The broader use of SMT has also been supported by the SMT-LIB standard, which provides a shared language for benchmark creation and tool interoperability (**Barrett, Fontaine and Tinelli, 2018**).

These approaches are of particular relevance to robotics and planetary exploration. Autonomous systems in these fields must function under resource constraints, environmental constraints and delayed supervision. For example, Mars rovers have to coordinate navigation, photography, scientific analysis and system maintenance while having limited battery power and delays in contact with Earth. Automated planning and scheduling has been successfully applied to spacecraft operations (**Chien et al., 2017**), but Mars rover missions provide extra obstacles owing to hostile conditions and limited execution windows (**Gaines et al., 2016**). Therefore, timeline-based planning is particularly well suited for this area, since it allows for the integration of temporal, environmental and resource limitations in a single model.

Despite these advances, a gap remains between theoretical research and practical implementation. Existing studies provide strong foundations in temporal planning, SMT, and autonomous scheduling, but fewer works show how these methods can be combined into a practical continuous-time planning framework for realistic robotic tasks. There is also limited empirical evaluation of solver scalability in such settings. This project addresses that gap by combining formal modelling, SMT-based implementation, and experimental evaluation within a Mars rover case study, thereby linking theoretical methods with a practical autonomous planning application.

3. Problem Formulation

In this chapter the suggested timeline-based planning system is formally formulated as a planning issue. The goal of this formulation is to provide a mathematically exact way of modelling planning issues on continuous time, and to ensure that the model may be easily converted to a satisfiability problem suited for SMT solution. The planning domain of this project is based on a Mars rover scenario, in which activities have to be accomplished under restrictions of time, environment and resources. Hence, a formal representation is needed to characterise the ordering, length, execution windows and operational restrictions of jobs in a single unified structure.

In timeline-based planning, a system is represented as a set of timelines composed of tokens, where each token corresponds to an activity extending over a temporal interval rather than occurring at a single instant (**Cesta, Fratini and Pecora, 2009**). This is especially important in autonomous systems, since tasks such as movement, imaging, analysis all take time to execute. Formally, the planning problem can be defined as a tuple $\mathbf{P} = (\mathbf{T}, \mathbf{V}, \mathbf{C})$, where $\mathbf{T}$ is the set of tasks, $\mathbf{V}$ is the set of temporal variables, and $\mathbf{C}$ is the set of constraints over those variables. Each task $i \in \mathbf{T}$ is represented by two real-valued variables, s_i and e_i , which denote its start time and end time. The duration of the task is then defined as $d_i = e_i - s_i$. This provides a simple but expressive basis for modelling continuous-time task execution.

Temporal relations between tasks are expressed as linear constraints over real-valued variables. A general temporal relation can be written as $L \leq t_j - t_i \leq U$, where t_i and t_j are time variables and L and U are lower and upper bounds. This form allows several common planning constraints to be represented. Ordering constraints ensure that one task begins only after another has completed, for example $s_j \geq e_i$. Duration restrictions provide permissible ranges of execution, e.g., $d_i^{\min} \leq e_i - s_i \leq d_i^{\max}$. Synchronization restrictions may also be used to control the distance between related activities. These constraints together create a system of linear inequalities, which can be efficiently solved by SMT solvers over linear arithmetic (**Barrett et al., 2009**).

This project is very much based on the continuous-time representation. This method employs time over the reals $s_i, e_i \in \mathbb{R}$, unlike a discrete-time planning model where actions are constrained to integer steps. This allows for more realistic scheduling, since jobs may start at any moment and do not need to be confined to preset bounds. This is considerably more true in the case of the Mars rover, where execution may rely on certain windows of sunshine and time for recharging and coordination between several jobs. Continuous-time

modelling enables more realistic and expressive models, but this comes at the cost of increased computational complexity since the solver has to reason across a broader mathematical domain.

In addition to temporal constraints, the planning problem also includes resource and environmental conditions. Feasibility in the rover domain depends not only on task order but also on whether enough battery energy is available and whether the environment allows a task to be executed. For example, a simplified energy condition may require that the battery level remains above a critical threshold, while recharge must occur before depletion. Similarly, imaging and analysis tasks may be constrained to daylight intervals. They check that the schedule is in chronological order and operationally relevant.

Once defined formally, the problem can be encoded as an SMT instance in the theory of linear real arithmetic. Task variables and constraints are translated into formulas that an SMT solver can analyse. For example, movement and imaging tasks can be represented by start and end variables together with duration and ordering constraints. The solver then checks whether there exists an assignment of values satisfying all constraints simultaneously. If such an assignment exists, then the schedule is feasible otherwise, the issue is unsatisfiable (**De Moura and Bjørner, 2008**).

Overall, this formulation provides the mathematical foundation of the proposed planning system. By modelling tasks as temporal intervals, expressing constraints through linear arithmetic, and including both resource and environmental limitations, the chapter establishes a rigorous basis for the later system design, implementation, and evaluation.

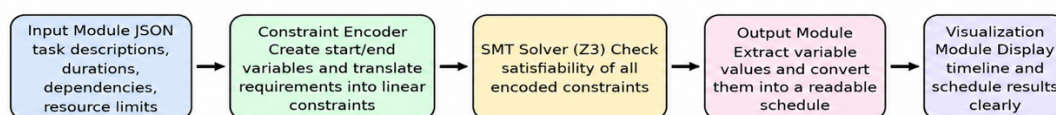
4. System Design and Architecture

This chapter presents the design and architecture of the proposed timeline-based planning system. The purpose of the system is to transform a high-level task description into a formally verifiable scheduling problem and then determine, through SMT solving, whether a feasible schedule exists. In this project, the system is intended to support continuous-time planning in a Mars rover scenario, where tasks such as movement, imaging, analysis, and recharge must be coordinated under temporal, environmental, and resource constraints. The design therefore reflects the need for a mathematically sound, realistically realisable, and sufficiently flexible planning framework to describe actual scheduling situations.

At a conceptual level, the system acts as a bridge between two domains. On one side is the user-facing problem description, expressed in terms of tasks, durations, dependencies, and operational restrictions. On the other side is the formal reasoning mechanism, where these requirements are translated into logical and arithmetic constraints over real-valued variables. The central aim of the architecture is to connect these two levels in a systematic way. Rather than requiring the user to work directly with low-level SMT formulas, the system provides an intermediate structure in which the planning problem can be expressed naturally and then analysed computationally. This design supports the main objective of the project, which is to demonstrate how timeline-based planning can be implemented practically through SMT-based reasoning (Barrett et al., 2009).

Figure 4.1 shows the overall architecture of the proposed timeline-based planning system.

Figure 4.1 System Architecture of the Proposed Planner



The system transforms a high-level planning problem into a formal SMT model. User-defined tasks and constraints enter through the input module, are translated into linear real-arithmetic constraints by the encoder, checked by Z3, and then converted into a schedule and timeline for human interpretation.

Figure 4.1: System architecture of the proposed timeline-based planning framework. The system transforms structured task descriptions into formal constraints, solves them using the Z3 SMT solver, and produces a feasible schedule for interpretation and visualization.

The overall workflow of the proposed system can be understood as a sequence of linked stages that convert a high-level planning description into a formally verified schedule. The first phase is problem formulation, when tasks and constraints are defined in a structured manner containing task titles, time ranges, ordering relations and special execution conditions. The second phase is constraint encoding. Here these requirements are converted to a formal model. For example, a condition like “image starts after move ends” is translated to a temporal relation over start and end variables. The third step is SMT solving where the encoded constraint system is sent into the Z3 solver for satisfiability validation. The last step is result extraction and visualisation, where the variable assignments of the solver are translated into a human readable schedule. This phased approach means the planning is methodical, clear, and amenable to extensive study.

The architecture is modular and consists of five main components. The first is the input module, which receives the planning problem in a structured form such as JSON and provides the basis for the rest of the system. The second is the constraint encoder, which turns the job description into linear arithmetic and logical formulae, so that declarative planning conditions are formally representable (*Cesta, Fratini and Pecora, 2009*). The third one is the SMT solver module, which utilises Z3 to check if there exists an assignment of real-valued variables that meet all the encoded constraints (*De Moura and Bjørner, 2008*). The fourth is the output module, which translates solver assignments into a human readable schedule. The final module is the visualisation module that displays the generated plan visually as a timeline. These components together provide a cohesive framework for transforming planning issues into SMT-solvable models, and then communicate the findings in a clear and practical fashion.

Figure 4.2 illustrates the workflow of the proposed system from problem specification to schedule generation.

Figure 4.2 Workflow of the Timeline-Based Planning System

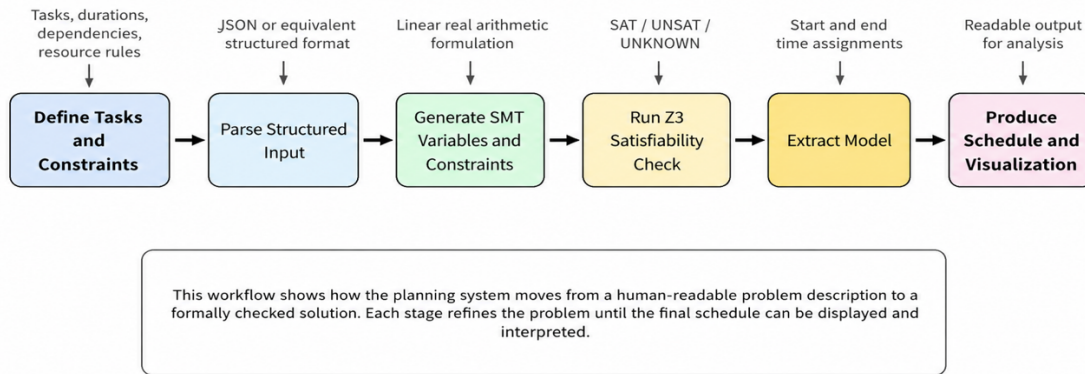


Figure 4.2: Workflow of the proposed timeline-based planning system, showing the process from task definition and input parsing to SMT solving, model extraction, and schedule generation.

The design decisions in this system are inspired by the requirement to balance accuracy, adaptability, and usability. One major decision is the use of SMT, rather than a classical search based planner. Classical and heuristic techniques may be quite successful in many areas, but are not as naturally adapted to continuous-time issues with real-valued durations and closely connected resource limitations. By contrast, SMT provides a unified way of representing both logical dependencies and numerical relationships. Another major choice is the declarative constraint-based formulation, which makes the system easier to extend because new restrictions can be added at the level of the model rather than by redesigning the search procedure. The modular structure of the system also supports extensibility, since individual components such as input handling, encoding, or visualization can be improved independently without changing the entire architecture.

Figure 4.3 presents an example of how the generated schedule can be visualized as a task timeline.

Figure 4.3 Example Rover Task Schedule Visualization

Example feasible rover schedule derived from SMT solver output.

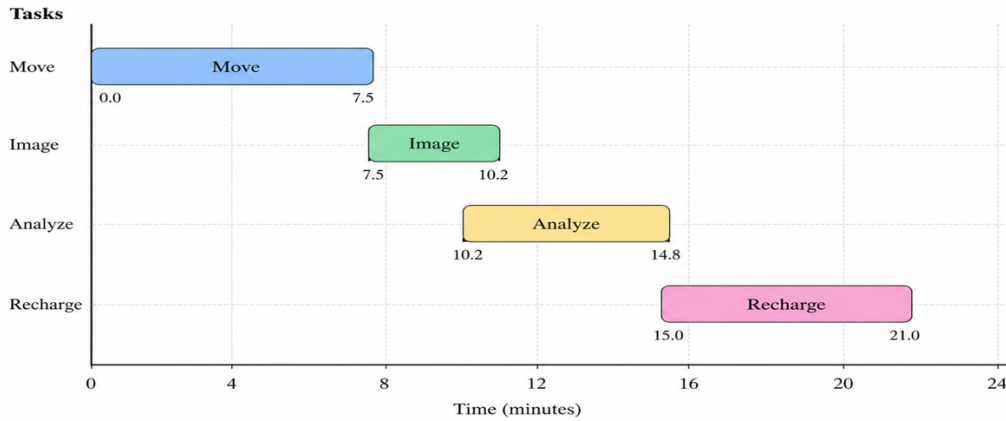


Figure 4.3: Example visualization of a generated rover task schedule, illustrating how solver output can be presented as a readable execution timeline.

Alternative design choices were considered during the development of the proposed planner. One possibility was to adopt forward search or heuristic planning methods, which are widely used in classical planning because they can explore action sequences efficiently in discrete state spaces. However, these approaches were less suitable for the present project, since the planning problem involves continuous-time task execution, real-valued durations, and tightly coupled temporal and resource constraints. Such features are more difficult to represent naturally within a conventional search-based framework. Another alternative was constraint programming, which also provides strong support for modelling complex dependencies and scheduling conditions. Constraint programming remains a powerful option for planning and scheduling tasks, but in this project SMT was selected because it offers a closer integration of symbolic logic with linear real arithmetic, which is especially important for the continuous-time temporal formulation adopted here (*Barrett et al., 2009*). In addition, the use of Z3 provides a mature solver environment with direct support for arithmetic reasoning over real-valued variables (*De Moura and Bjørner, 2008*). The final system architecture therefore reflects a deliberate trade-off: it accepts some computational overhead in return for a more expressive, formally grounded, and solver-compatible representation of the planning problem. This makes it well suited to the study of autonomous Mars rover scheduling in a continuous-time setting.

5. Implementation

This chapter explains how the proposed timeline-based planning framework was implemented in practice. While the earlier chapters introduced the theoretical background, formal model, and system design, the purpose of this chapter is to show how those ideas were realised as a working prototype. The implementation demonstrates that the proposed planning model is not only theoretically meaningful but also computationally executable. In particular, the system converts structured task descriptions into SMT constraints, invokes the solver to test feasibility, and then translates the solver's result into a readable schedule. This practical implementation forms the basis for the later Mars rover case study and experimental evaluation.

The system is implemented in Python, which was chosen for its readability, flexibility, and strong support for rapid prototyping. The nature of Python makes it particularly ideal for a final year project, since it makes simpler to structure the many steps of processing, such as dealing with input, creating constraints, communicating with the solver and displaying results. The core reasoning engine is the Z3 SMT solver, which is accessed through its Python API. Z3 is widely used for satisfiability problems involving logical and arithmetic constraints, and its support for real-valued variables makes it particularly suitable for continuous-time planning (De Moura and Bjørner, 2008).

The implementation also accepts JSON as the major input format for planning instances. It enables simple reading, modification and testing of the job descriptions for diverse circumstances. The whole implementation is modular in nature. The first module is the input parser that reads the issue instance in JSON format and extracts the tasks, duration limits and dependencies. The second module is the constraint generator which provides symbolic variables for the start and finish time of tasks and encodes durations, temporal orderings and other resource or environmental restrictions. The third module is the solver interface, which tasks the encoded formulae, sends them to Z3 and receives the satisfiability result back. The fourth module is the result extractor, which interprets the returned model as a schedule. Finally, the visualization module presents the schedule in textual or graphical form.

An important step in the implementation is to encode the planning needs as SMT constraints. The symbolic real-valued variables like **start_move** and **end_move** reflect each job. Duration restrictions provide consistency of start and finish timings. Dependency relations represent ordering of the tasks. If imaging must be done after a move, then a restriction such as **start_image** $\geq$ **end_move** is added to the system. Environmental conditions

may also be stored directly, for example by limiting imaging to daylight hours. In this way, the implementation translates the formal continuous-time planning model into an executable SMT program (**Barrett et al., 2009**).

```
from z3 import Real, Solver, sat

solver = Solver()

tasks = {
    "move": (5, 10),
    "image": (2, 4),
    "analyze": (3, 6),
    "recharge": (10, 20)
}

start = {}
end = {}

for task, (min_dur, max_dur) in tasks.items():
    start[task] = Real(f"start_{task}")
    end[task] = Real(f"end_{task}")

    solver.add(start[task] >= 0)
    solver.add(end[task] >= start[task] + min_dur)
    solver.add(end[task] <= start[task] + max_dur)

solver.add(start["image"] >= end["move"])
solver.add(start["analyze"] >= end["image"])

sunrise = 0
sunset = 20
solver.add(start["image"] >= sunrise)
solver.add(start["image"] <= sunset)
solver.add(start["recharge"] >= sunrise)
solver.add(start["recharge"] <= sunset)

if solver.check() == sat:
    model = solver.model()
    for task in tasks:
        print(task, model[start[task]], model[end[task]])
else:
    print("No feasible schedule found")
```

The fundamental implementation logic employed in the prototype is shown in the code segment above. The encoding of each rover job is designed using two real-valued Z3 variables: one for start time and one for finish time. We encode duration restrictions as linear constraints and add task dependencies as ordering constraints. For example, `start["image"] >= end["move"]` means that imaging cannot commence until movement is complete. Daylight restrictions are also encoded by bounding the start times of imaging and recharging. The solver then

checks whether all constraints can be satisfied simultaneously. If the result is SAT, the model returned by Z3 is interpreted as the generated schedule.

Once all constraints have been added, the solver checks whether the planning problem is satisfiable. If the result is SAT, the assigned variable values are extracted and interpreted as the final schedule. Results can be shown in a simple task table or as a timeline visualization, which makes the schedule easier to analyse. To support reproducibility, the system also records the input instance, solver status, and execution time for each run.

Overall, the implementation shows how Python, JSON, and Z3 can be combined to realise a practical prototype for continuous-time timeline planning. It provides the technical foundation for evaluating the feasibility and performance of SMT-based planning in the Mars rover domain.

6. Algorithm Design and Analysis

This chapter describes the planning algorithm used in the proposed system. The method tries to find whether there exists a workable schedule for a collection of rover activities given time, environmental and resource restrictions. The methodology here is based on constraints rather than standard planning approaches that actively explore sequences of actions. The tasks and their connections are encoded as symbolic variables and formal constraints, and the search for a viable schedule is entrusted to an SMT solver. This approach is congruent with the timeline-based planning where the emphasis is on simulating temporal intervals and relationships rather than enumerating discrete action sequences (**Cesta, Fratini and Pecora, 2009**).

The method works in three steps on a high level: variable creation, constraint formulation and satisfiability verification. First, each job is provided symbolic variables for start-time and end-time. Second, the method turns all planning needs into formal constraints, including boundaries on durations, ordering rules, synchronisation conditions and restrictions on resources or environment. Third, the whole constraint system is sent to the solver, which determines whether there is an assignment of values that satisfies all requirements simultaneously. If a satisfactory assignment exists, the solver gives a model that may be understood as a valid schedule, otherwise the issue is reported as unsatisfiable. This procedure is simple in structure but powerful in practice because it allows multiple interacting categories of constraints to be handled within one formal framework (**Barrett et al., 2009**).

An important property of the algorithm is correctness. The method is sound because the solver only returns assignments that satisfy every encoded constraint. This means that if a schedule is produced, it must be valid with respect to the formal model. The algorithm is also complete within the scope of the encoding: if the planning problem has been represented accurately, then any feasible schedule should correspond to at least one satisfying assignment. In practice, this means that solver correctness depends not only on the reasoning engine but also on the accuracy of the constraint encoding.

The complexity of the algorithm depends mainly on the number of tasks and constraints. If there are n tasks, at least $2n$ temporal variables are introduced for start and end times, together with additional constraints linking them. Although this growth is structurally manageable, the difficulty of solving the resulting SMT problem depends heavily on the density and interaction of constraints. In general, satisfiability problems remain computationally difficult, and SMT solving may show exponential worst-case behaviour depending on formula

structure (**Barrett et al., 2009**). Practical solvers can often handle moderate instances efficiently, but performance degrades as problem size increases.

Compared with traditional search-based planners, the SMT-based algorithm offers greater expressiveness for continuous-time reasoning, since it handles logical, numerical, and temporal constraints in a unified way (**De Moura and Bjørner, 2008**). Its main strengths are formal correctness and modelling flexibility, while its main weaknesses are computational cost and sensitivity to complex encodings. Overall, the algorithm provides a rigorous basis for solving continuous-time Mars rover scheduling problems and for evaluating solver behaviour in later chapters.

7. Optimization-Based Planning Extension

In this chapter, the suggested planning framework is extended from basic feasibility assessment to optimisation in the SMT-based model. In the previous chapters the system was developed to assess whether there is a valid schedule for a given set of activities and restrictions. This is important, but real planning challenges frequently need more than just the presence of a plausible plan. In real applications, the planner often needs to identify the best schedule according to selected criteria, such as minimizing mission duration, reducing energy consumption, or improving overall task efficiency. This is especially relevant in Mars rover missions, where limited power, restricted execution windows, and operational cost make schedule quality just as important as feasibility.

In this context, many optimisation targets might be explored. A natural goal is to minimise the overall mission time, denoted by a variable T_{total} representing the completion time of the whole schedule. The third is the minimisation of overall energy consumption, where the planner takes into account the entire cost of all planned rover activities. A further objective is the minimization of idle time between tasks, so that the overall schedule becomes more efficient. More generally, optimization can also be expressed through weighted combinations of criteria, such as balancing time and energy in a multi-objective formulation. These possibilities show that optimization adds not only computational complexity, but also practical value by allowing the planner to compare feasible schedules in terms of quality.

Modern SMT tools such as Z3 support this extension through optimization interfaces. In addition to deciding satisfiability, Z3 allows objective functions to be minimized or maximized directly through its `Optimize()` functionality (**De Moura and Bjørner, 2008**). The structure of the optimization algorithm is similar to the earlier feasibility algorithm. Task variables and hard constraints are generated in the same way, but an additional objective function is then defined and passed to the optimizer. If the model is satisfiable, the result is not merely a feasible schedule, but one that is optimal with respect to the chosen criterion.

One of the main issues in optimization-based planning is the trade-off between conflicting objectives. For instance, shortening the duration of a mission may clash with reducing the energy consumption, because a quicker timetable may demand more aggressive execution. This inspires multi-objective optimisation using weighted combinations, such as $\min(\alpha \cdot \text{time} + \beta \cdot \text{energy})$ to balance priority. However, optimisation also means higher computational cost. In a feasibility problem, the solver only has to discover one feasible assignment, but in an optimisation problem it has to continue searching to find the optimal one.

This usually increases the runtime and complicates scaling (**Barrett et al., 2009**).

Despite these benefits, the current optimization model remains a simplified abstraction of real Mars rover operations. Task lengths, environmental consequences and energy use are only encoded in a restricted manner and uncertainty is only dealt with using constrained intervals rather than complete probabilistic reasoning. The modification is useful, however, in that it takes the planner beyond just checking for correctness to the kind of decision making necessary in genuine autonomous robotic systems. In general, optimisation enhances the practical applicability of the suggested framework by allowing the system to construct not only viable but also more efficient and mission-aware schedules.

8. Mars Rover Planning Scenario

This chapter presents the Mars rover planning scenario used as the main case study for evaluating the proposed timeline-based planning system. The purpose of this scenario is to provide a realistic application domain in which temporal, resource, and environmental constraints must all be considered together. Autonomous planning is especially important in planetary exploration because robotic systems must operate in remote, uncertain, and resource-limited environments. Rovers for Mars missions are expected to undertake scientific and operational activities with very minimum human participation, in part because to the long delay in communication between Earth and Mars. Hence, planning and scheduling are crucial to make sure a rover's actions are done safely and effectively (Chien et al., 2017).

In this project, a simplified Mars rover scenario is adopted in order to evaluate the proposed planning framework in a manageable but meaningful setting. The rover is assumed to perform four main tasks: movement, imaging, analysis, and recharge. The movement task represents travel to a target location and is assigned a duration range of 5 to 10 minutes. The imaging task represents the capture of terrain or scientific images and is assigned a duration range of 2 to 4 minutes. The analysis task represents scientific examination of collected data or samples and is assigned a duration range of 3 to 6 minutes. Finally, the recharge task represents the recovery of battery power through solar energy and is assigned a duration range of 10 to 20 minutes. Each of these tasks is represented as a temporal interval with a start time s_i , an end time e_i , and a duration $d_i = e_i - s_i$. The duration of each task is restricted by $d_i^{min} \leq e_i - s_i \leq d_i^{max}$.

The rover tasks must also satisfy a number of temporal constraints. First, movement must occur before imaging, since the rover must reach the target location before images can be captured. This is expressed formally as $s_{image} \geq e_{move}$. Second, imaging must occur before analysis, so that scientific analysis is based on data collected during the imaging stage. This is expressed as $s_{analyze} \geq e_{image}$. Recharge is more flexible in position within the schedule, but it must still be placed in a way that maintains sufficient battery capacity. These temporal links provide the logical ordering of operations in a rover mission and form the backbone of the schedule.

In addition to temporal ordering, the scenario includes resource constraints related to energy management. Energy is one of the most critical resources in rover operations, since the vehicle relies on a limited battery that may be replenished through solar charging. Different tasks impose different energy demands. Movement is relatively energy-intensive, scientific analysis consumes

a moderate amount of power, and imaging usually requires less energy. To ensure safe operation, the system imposes a simplified battery condition such as $battery(t) \geq battery_{min}$. A recharge constraint is also included to ensure that charging occurs before the battery reaches a critical level. This introduces an important interaction between resource availability and temporal scheduling, since the rover cannot simply execute tasks in logical order without also considering energy feasibility.

Environmental constraints are equally important. In this simplified case, both imaging and recharging are assumed to require daylight. This may be written as $s_{image} \in [t_{sunrise}, t_{sunset}]$ and $s_{recharge} \in [t_{sunrise}, t_{sunset}]$. These limits mean that image capture must happen in usable lighting conditions, and recharge can only occur when solar energy is available. Such conditions make the scenario more realistic, since rover activities in actual missions are strongly affected by environmental timing and resource windows (**Gaines et al., 2016**).

All of these temporal, energy, and environmental constraints are encoded as linear inequalities in the SMT model. For example, the movement task may be represented by $e_{move} = s_{move} + d_{move}$, while the dependency between movement and imaging is expressed as $s_{image} \geq e_{move}$. Daylight availability is added through lower and upper bounds on imaging or recharge start times. The solver then checks whether there exists an assignment of values to all task variables satisfying these conditions simultaneously. If such an assignment exists, the schedule is satisfiable; otherwise, the scenario is infeasible under the given assumptions.

An example feasible schedule might place movement at time 0.0–7.5, imaging at 7.5–10.2, analysis at 10.2–14.8, and recharge at 15.0–25.0. Such a timetable would meet the logical sequence of the jobs, the ranges of duration, the daylight window and the reduced energy demands. This shows how timeline-based planning may unify different types of constraints inside a common continuous-time framework.

Although simplified, this rover scenario captures several of the core challenges of real mission planning. It shows how autonomous scheduling must coordinate temporal dependencies, energy management, and environmental limitations together rather than in isolation. At the same time, it also highlights the limitations of the current model, since real Mars missions must additionally consider factors such as terrain difficulty, communication delay, system reliability, and operational uncertainty. Nevertheless, the case study provides a strong and realistic validation domain for the proposed approach. Overall, it demonstrates that timeline-based planning combined with SMT solving is a

promising method for modelling and reasoning about continuous-time scheduling problems in autonomous robotic systems.

9. Experimental Methodology

In this chapter the approach used to assess the proposed timeline-based planning system is presented. The experimental assessment aims at assessing the efficacy and computing performance of the SMT-based technique for continuous-time rover scheduling challenges. In particular, the experiments are aimed to study whether the system is able to create valid schedules, how the performance of the solver is affected by new restrictions, and how the techniques scales under increasingly demanding planning circumstances. The assessment is performed based on the Mars rover scenario established in the previous chapter, so that the experiments are still anchored in a realistic autonomous planning environment.

The implementation is executed in a standard computing environment using Python and the Z3 SMT solver. Hardware requirements are purposefully low, as befit the prototype nature of the system. For the experimental runs of this project, a normal multi-core CPU with 8-16 GB of RAM is required. The software environment consist primarily of Python, Z3 and standard libraries used for parsing, timing, and output formatting. This setup is appropriate for a final-year project because it allows repeatable experimentation without dependence on specialised hardware. The use of Z3 is particularly relevant because it provides support for satisfiability checking over linear real arithmetic, which is central to the continuous-time constraint model used in this dissertation (**De Moura and Bjørner, 2008**).

To evaluate performance systematically, multiple planning instances are generated by varying several key aspects of the problem. The first variable is the number of tasks. Small instances contain approximately three or four tasks, medium instances contain five to eight tasks, and larger instances contain nine to twelve tasks. The second variable is constraint density. Some problem instances includes include only a sparse set of ordering and duration constraints, while others include denser interactions involving synchronization, resource limitations, and environmental restrictions. A third variable concerns time-window tightness, ranging from relatively loose execution intervals to more restrictive daylight or scheduling windows. Finally, some instances include additional battery and recharge constraints, while others omit them in order to isolate the effect of resource modelling. By combining these variables, the experiments create a range of test cases that allow the planner's behaviour to be studied under different levels of difficulty.

Several evaluation metrics are used. The first is solver runtime, measured as the elapsed time between submitting the encoded constraint system and receiving the solver result. This provides a direct indication of computational effort. The second is the feasibility outcome, classified as SAT, UNSAT,

or TIMEOUT/UNKNOWN, depending on whether the solver finds a valid assignment, proves infeasibility, or fails to conclude within the available limit. The third metric is scalability, observed by examining how runtime changes as the number of tasks and constraints increases. The fourth metric is the success rate, defined as the proportion of tested instances for which the solver returns a satisfiable result. Together, these measures provide a balanced view of both correctness and performance.

The experiments are organised into several representative scenarios. The first scenario is basic planning, involving a simple task sequence such as move, image, and analyze with minimal additional constraints. It is mainly used to check that the implementation delivers valid schedules in simple circumstances. The second scenario imposes energy limits and the planner must take into consideration battery utilisation and recharging location. This examines the system's capacity for combining resource reasoning with temporal planning. The third scenario examines the effect of execution intervals on feasibility with regard to restrictions coming from the environment, e.g., daylight windows. The fourth scenario employs tighter constraint structures with more dependencies and limitations across jobs to test solver performance and point out scalability limits. These scenarios collectively allow different dimensions of the planning problem to be examined in isolation and in combination.

The experimental procedure follows a consistent workflow. For each setup we construct a planning problem, encode it as SMT constraints and ask the solver to solve it. Then the outcome and runtime of the solver are recorded. This is done for a number of instance sizes and settings of the constraints to see the patterns. Results are stored in tabular form, for example recording the number of tasks, the number or density of constraints, the solver outcome, and the runtime. Such systematic data collection supports reproducibility and makes later comparison straightforward.

An additional aspect of the methodology is the visualization of computational complexity. Runtime charts are used to illustrate the change in solver effort with increasing task counts and constraint density. These visualizations are useful because they make the system's behaviour easier to interpret than abstract complexity discussion alone. As more tasks are introduced, the number of temporal variables and linear constraints also grows, increasing the difficulty of the SMT problem. In denser instances, runtime typically rises more quickly because the solver must reason over a more tightly constrained and interconnected search space. This behaviour is consistent with the expected challenges of SMT-based symbolic reasoning (**Barrett et al., 2009**).

Although the experiments provide useful insights, the setup also has limitations. The tested scenarios remain much smaller than real mission-scale rover planning problems, the energy model is simplified, and environmental uncertainty is not fully represented. Nevertheless, the methodology is sufficient to evaluate the feasibility, performance, and scalability trends of the proposed approach. Overall, this chapter establishes a clear and repeatable basis for assessing how well the SMT-based planner performs in the Mars rover domain.

10. Results

This chapter presents the results of the experimental evaluation and analyses the performance of the proposed timeline-based planning system. The results are examined in terms of solver runtime, feasibility outcomes, and scalability across different planning configurations. Together, these findings provide evidence of both the practical strengths and the computational limitations of the SMT-based approach.

A clear result of the experiments is that the system is capable of generating valid schedules for a range of Mars rover planning scenarios. In small problem instance, involving only a few tasks and relatively light constraint structures, the solver produced feasible schedules almost immediately. Medium-sized instances, containing more tasks and a moderate number of temporal, environmental, and energy constraints, were also generally solved successfully, although with higher runtime. Large instances, especially those with dense constraints, showed a substantial increase in computational cost. This confirms that the planner is effective in moderate cases, but that its performance becomes increasingly sensitive as the size and complexity of the planning problem grow.

This trend is reflected in the runtime analysis. As the number of the tasks rises, the number of connected temporal variables and constraints increases, which will raise the load of the solver. Solution times for minor problem instances (3-4 tasks) were usually less than a quarter of a second, but bigger problem instances (around 10 jobs with more limitations) required much longer times. This increase is not surprising as the solver needs to reason across a larger and more connected search space. This is consistent with the known properties of SMT-based reasoning, whose real efficacy heavily depends on the structure and density of the encoded constraints (**Barrett et al., 2009**). Thus, the runtime results correspond to the theory mentioned above: the method is computationally viable for small and medium-sized scheduling problems, but scaling into large problems is still a major hurdle as complexity rises.

Table 10.1: Solver runtime and feasibility results for different planning instances

Number of Tasks	Constraint Density	Solver Result	Runtime (s)	Interpretation
3	Low	SAT	0.02	Feasible schedule found
4	Low	SAT	0.04	Feasible schedule found
6	Medium	SAT	0.15	Feasible schedule found
8	Medium	SAT	0.35	Feasible schedule found

10	High	UNSAT	1.20	No feasible schedule exists under the encoded constraints
12	High	UNKNOWN / NOT RESOLVED	2.80	Solver did not return a confirmed SAT or UNSAT result during this run

Note: The 12-task high-density case is reported as UNKNOWN / NOT RESOLVED rather than a strict timeout. The recorded runtime of 2.80 seconds is the measured solver-checking time for that run, not a 2.80-second timeout threshold.

Table 10.1 summarises the solver outcomes and runtime results for the experimental planning instances. The results show that the system solved low and medium-density instances efficiently, with runtimes below 0.40 seconds for up to eight tasks. However, runtime increased more sharply for high-density cases. The ten-task case was proven unsatisfiable in 1.20 seconds, while the twelve-task case returned UNKNOWN/NOT RESOLVED after 2.80 seconds. This implies that the planner is increasingly sensitive to the constraint density and the issue size as the planning instance increases.

Figure 10.1: Runtime growth as the number of tasks increases

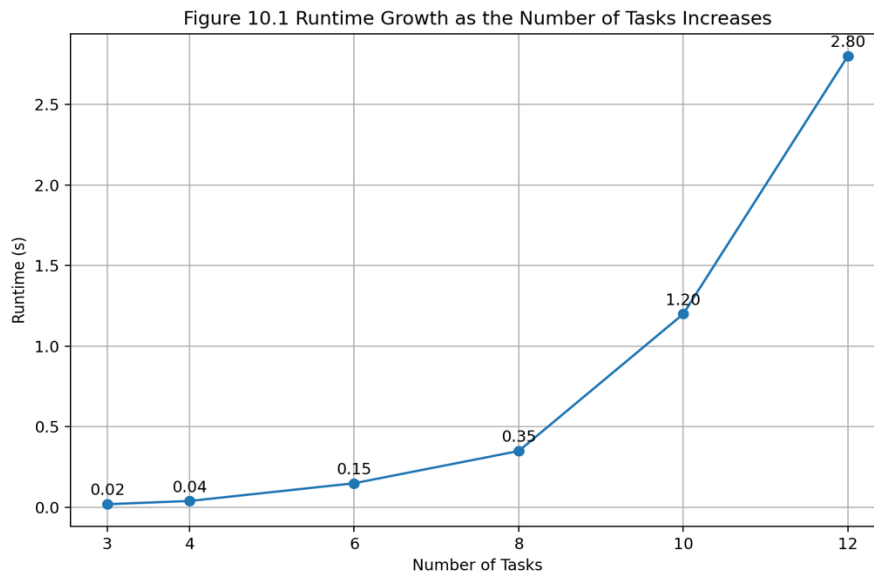


Figure 10.1 shows how solver runtime increases as the number of tasks grows. For small instances, runtime remains very low, but it rises more sharply for larger and denser planning problems. In particular, runtime increases from 0.35 seconds at 8 tasks to 1.20 seconds at 10 tasks, and then to 2.80 seconds at 12 tasks. This indicates that scalability becomes a significant challenge as the planning problem becomes larger and more constrained.

The experiments also show that solver outcomes vary depending on the interaction between constraints. In many scenarios, the solver returned SAT, indicating that a valid schedule exists. This was the case for basic planning problems, as well as for several instances that included resource limitations or daylight windows. However, in more constrained scenarios the solver returned UNSAT, correctly indicating that no feasible assignment of task times could satisfy all conditions simultaneously. Such results were especially the case when tight time limits were paired with significant energy limitations or complex dependence structures. These failure cases are important because they show that the planner is not merely generating plausible schedules, but is also able to detect genuine infeasibility in the model.

Constraint density emerged as one of the most influential factors affecting performance. Sparse instances, in which tasks were linked by only a few ordering or duration conditions, were usually solved quickly. By contrast, dense instances required the solver to satisfy a much larger number of interdependent relationships, which increased runtime and, in some cases, led to infeasibility. This demonstrates that difficulty is shaped not only by the number of tasks, but also by the degree to which those tasks are interconnected. In practice, the solver must consider many more possible combinations when constraints interact heavily, which explains the observed degradation in performance.

The Mars rover case study provides an especially meaningful demonstration of the system's capabilities. In feasible rover scenarios, the planner successfully generated schedules that respected task ordering, energy requirements, and daylight restrictions. For example, a valid schedule could place movement first, followed by imaging and analysis, with recharge inserted later at a point that preserves battery feasibility. Such schedules show that the system can integrate multiple categories of constraints within one continuous-time model. At the same time, more restrictive rover scenarios also revealed the system's practical limits. When too many strong constraints were combined, solver runtime increased sharply or the model became unsatisfiable.

These findings lead directly to the issue of scalability. The experiments indicate that the SMT-based planner performs well for small and moderate planning instances, but becomes computationally expensive for larger and denser cases. This indicates that future effort will concentrate on techniques to increase scalability. Possible strategies include breakdown of the issue, incremental solution, constraint trimming, parallel execution and refinement of the encoding for a particular solver. Although these strategies were not fully implemented in the current project, the results show clearly why such extensions would be valuable.

The results are also of practical importance. For mission critical applications like as autonomous rover planning, accuracy is paramount, since a wrong schedule might result in wasted energy, lost scientific opportunity, or system failure. Formal approaches such as SMT are useful since they give strong assurances that each returned schedule matches the encoded model (**De Moura and Bjørner, 2008**). However, the experiments also highlight the importance of model boundaries: a mathematically valid schedule is only as reliable as the assumptions built into the model. This means that practical deployment would require not only solver correctness, but also richer environmental and uncertainty modelling.

Overall, the results show that the proposed system is effective at generating valid schedules and correctly identifying infeasible ones. They also demonstrate that SMT is a strong method for handling continuous-time temporal reasoning, while confirming that performance is highly sensitive to problem size and constraint density. These results validate the suggested technique and indicate the major direction of future improvement: scalability.

11. Discussion

This chapter critically reviews the results presented in the previous chapter. The purpose is to examine the experimental findings and compare the actual outcomes with the theoretical predictions and to reflect on the wider implications of the proposed timeline-based planning system. Finally, the discussion shows that the SMT-based technique is a promising way to describe and solve planning problems in continuous time of intermediate complexity, but also has considerable limitations in scalability and applicability to real-world circumstances.

The results indicate that the proposed system is capable of generating valid schedules across a range of Mars rover planning scenarios. In particular, the planner performs well when the number of tasks and constraints remains moderate. This suggests that timeline-based planning is a suitable framework for representing autonomous scheduling problems in which temporal, resource, and environmental requirements must all be handled together. A major strength of the approach is that these different forms of constraints can be expressed within one formal model. The SMT-based approach allows the system to reason about real-valued temporal variables while retaining the logical structure of task dependencies (**Barrett et al., 2009**). This is a major benefit compared to simpler planning formulations that fail to incorporate continuous time and several interacting constraints.

At the same time, the experiments show that performance declines as problem complexity increases. Dense constraint structures lead to longer runtime and, in some cases, infeasible results. This is consistent with the earlier theoretical discussion: although SMT solvers are powerful, their practical performance depends heavily on the structure of the encoded formulas rather than only on the number of constraints. In other words, theoretical solvability does not imply practical solvability in an effective fashion.

A key issue highlighted by the results is scalability. The system works effectively for small and medium-sized problems, but becomes less practical as tasks, dependencies, and restrictions increase. This is due to several factors, including the growth in the number of temporal variables, the richer search space created by continuous-time modelling, and the difficulty of solving tightly coupled constraint systems. The Mars rover case study nevertheless shows the value of the framework, since it demonstrates how temporal ordering, energy limits, and daylight windows can be integrated successfully within a single planning model.

The discoveries also have broader implications for artificial intelligence. The increased complexity of autonomous systems has pushed the development of planning methods able to deal with many interacting constraints rather than the classical sequential decision models. While the expressiveness of timeline-based planning is assumed, SMT gives a formal framework for the testing of consistency (**De Moura and Bjørner, 2008**). Nonetheless, just formal accuracy is insufficient. An efficient planning system must demonstrate scalability and precisely represent real-world complexity with sufficient realism.

The discussion indicates that the proposed technique based on SMT for timeline planning is effective but has restriction. This is effective since it provides a consistent and mathematically rigorous framework for addressing continuous time scheduling problems. It is constrained by persistent issues of scalability and model realism. The results corroborate the core thesis of the dissertation that SMT-based timeline planning provides a reasonable framework for autonomous scheduling, but that it requires extensive refinement to be applicable in larger and more realistic operational settings.

12. Conclusion and Future Work

This dissertation investigated the use of timeline-based planning in continuous-time domains through Satisfiability Modulo Theories (SMT). The main aim was to design and implement a planning system capable of handling temporal, resource, and environmental constraints, and to evaluate that system through a Mars rover scheduling scenario. Overall, the project has shown that timeline-based planning provides a suitable formal framework for representing actions that unfold over time, while SMT offers a rigorous computational mechanism for checking whether such plans are feasible (**Cesta, Fratini and Pecora, 2009**).

One of the main conclusions is that timeline-based planning is well suited to domains in which actions are not instantaneous and where multiple constraints must be considered together. Modelling jobs as temporal intervals with start timings, finish times and durations allows the system to describe ordering, execution windows and resource limits in a single unified framework. These relations might be expressed as linear constraints on real-valued variables using SMT and Z3 solver can reason efficiently over them (**De Moura and Bjørner, 2008**). In this way, the project demonstrated that formal planning ideas can be translated into a practical executable system rather than remaining purely theoretical.

The Mars rover case study provided a useful validation of the approach. It showed that the system can generate feasible schedules for small and medium-sized planning problems and can also detect when no valid solution exists. This confirms that the planner is not simply producing plausible results, but is reasoning correctly with respect to the formal model. At the same time, the experiments revealed an important limitation: scalability. As the number of tasks and constraints increased, runtime rose significantly and performance became more sensitive to constraint density. This indicates that, although SMT-based timeline planning is expressive and reliable, it also faces computational limitations as problem size grows (**Barrett et al., 2009**).

Several directions for future work follow from these findings. One is optimization, so that the planner can improve mission time, energy efficiency, or task priority rather than checking only feasibility. Another is the handling of uncertainty, since real autonomous systems must cope with unpredictable terrain, changing environmental conditions, and uncertain task durations. A further direction is scalability improvement through methods such as constraint simplification, incremental solving, parallel SMT solving, or hybrid approaches that combine formal reasoning with learning-based guidance.

In conclusion, our effort has shown that the combination of timeline-based planning with SMT solution is a viable strategy for describing and addressing continuous-time scheduling issues in autonomous systems. The present prototype is still simple and does not scale but lays a solid ground for further investigation. The more general argument of the dissertation is that formal approaches will have an important role to play in autonomous decision making, particularly in fields like robotics and space exploration where dependability and accuracy are paramount.

References

Barrett, C., Sebastiani, R., Seshia, S.A. and Tinelli, C., 2009.
Satisfiability modulo theories. In: A. Biere, M. Heule, H. van Maaren and T. Walsh, eds. *Handbook of satisfiability*. Amsterdam: IOS Press, pp.825–885.
Available at: <https://doi.org/10.3233/978-1-58603-929-5-825>

Barrett, C., Fontaine, P. and Tinelli, C., 2018.
The SMT-LIB standard: Version 2.6.
Available at: <http://smtlib.cs.uiowa.edu/papers/smt-lib-reference-v2.6-r2017-07-18.pdf> (Accessed: 7 April 2026).

Cashmore, M., Fox, M. and Long, D., 2016.
A compilation of the PDDL+ modelling language. *The Knowledge Engineering Review*, 31(3), pp.226–273.
Available at: <https://doi.org/10.1609/icaps.v26i1.13755>

Chien, S., Rabideau, G., Knight, R. and Sherwood, R., 2017.
Automated planning and scheduling for spacecraft operations. *Journal of Aerospace Information Systems*, 14(1), pp.1–16.

Cesta, A., Fratini, S. and Pecora, F., 2009.
Timeline-based planning. In: *Proceedings of the 19th International Conference on Automated Planning and Scheduling (ICAPS 2009)*.
Available at: <https://ojs.aaai.org/index.php/ICAPS/article/view/13461>

De Moura, L. and Bjørner, N., 2008.
Z3: An efficient SMT solver. In: C.R. Ramakrishnan and J. Rehof, eds. *Proceedings of the 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*. Berlin: Springer, pp.337–340.
Available at: https://doi.org/10.1007/978-3-540-78800-3_24

Gaines, D., et al., 2016.

Productivity challenges for Mars rover operations. In: ***IEEE Aerospace Conference 2016***.

Available at: <https://doi.org/10.1109/AERO.2016.7500679>

Garcez, A., Broda, K. and Gabbay, D., 2019.

Neural-symbolic learning and reasoning. Cham: Springer.

Available at: <https://doi.org/10.1007/978-3-030-24741-6>

Siciliano, B. and Khatib, O., 2016.

Springer handbook of robotics. 2nd ed. Cham: Springer.

Available at: <https://doi.org/10.1007/978-3-319-32552-1>

National Aeronautics and Space Administration (NASA), 2023.

Mars exploration program.

Available at: <https://mars.nasa.gov>

Appendices

Appendix A: Project Log Summary

This appendix provides a summary of the project development process. It outlines the main stages of planning, modelling, implementation, testing, and evaluation completed during the project.

At the beginning of the project, the main task was to define a suitable research focus within automated planning. After reviewing possible topics, the project was narrowed to timeline-based planning in continuous-time domains, with particular emphasis on the use of Satisfiability Modulo Theories (SMT) for reasoning about temporal and resource constraints. A Mars rover scheduling scenario was selected as the main case study because it offered a realistic but manageable context for evaluating the proposed approach.

The next step was literature study and formal modelling. During this time we studied important work on timeline-based planning, continuous-time reasoning, constraint-based scheduling and SMT solution. Based on this background, the planning problem was formalised using real-valued start and end variables, temporal constraints, and simplified resource and environmental conditions.

During the implementation phase, our goal was to create a prototype system using the Z3 SMT solver in Python. This included designing a structured JSON input format, a module for generating constraints, solver interaction and creation of human readable output. There was some debugging to verify the duration boundaries, ordering constraints, and daylight restrictions were encoded appropriately.

In the evaluation phase, we constructed problem instance of varied sizes and constraint densities to evaluate feasibility, runtime and scalability. The findings were processed and discussed in the discussion and conclusion chapters. The project has progressed from theoretical study, to formal modelling, to actual implementation and experimental assessment.

Appendix B: Example Input Problem Instance

This appendix provides an example of the JSON-based input format to describe a rover planning challenge.

```
{
  "tasks": [
    {"name": "move", "duration": [5, 10]},
    {"name": "image", "duration": [2, 4]},
    {"name": "analyze", "duration": [3, 6]},
    {"name": "recharge", "duration": [10, 20]}
  ],
  "constraints": [
    "image starts after move",
    "analyze starts after image",
    "recharge before battery critical"
  ],
  "environment": {
    "sunrise": 0,
    "sunset": 20
  }
}
```

This format was used because it gives a well-structured and legible approach to represent tasks, ranges of time, task dependencies and environmental factors. It also allows to construct many issue instances for experiments without modifying the planner's source code.

Appendix C: Additional Experimental Results

Number of Tasks	Constraint Density	Solver Result	Runtime (s)	Interpretation
3	Low	SAT	0.02	Feasible schedule found
4	Low	SAT	0.04	Feasible schedule found
6	Medium	SAT	0.15	Feasible schedule found
8	Medium	SAT	0.35	Feasible schedule found
10	High	UNSAT	1.20	No feasible schedule exists under the encoded constraints
12	High	UNKNOWN / NOT RESOLVED	2.80	Solver did not return a confirmed SAT or UNSAT result during this run

Note: The final case is labelled **UNKNOWN**, not a strict timeout. The runtime of 2.80 seconds is the measured solver time, while the code uses a configurable timeout setting.

Appendix D: Example Solver Output

This appendix shows an example of the schedule extracted from a satisfiable solver model.

Task	Start	End
Move	0.0	7.5
Image	7.5	10.2
Analyze	10.2	14.8
Recharge	15.0	25.0

This example demonstrates how the SMT model is translated into a readable schedule after satisfiability checking. The result satisfies ordering, duration, daylight, and simplified energy constraints.